Tarea 3: Neo4j y análisis de grafos

Integrantes: Karina Aliaga Romero, 202373556-6
Nicolás Chehade Casivar, 202373508-6
Fecha: 1 de noviembre de 2026

1. Instalación y verificación

Se instaló GDS en la instancia de Neo4j y tras la ejecución del comando `RETURN gds.version() AS versionGDS;` se obtuvo como resultado la versión 2026.10.1, confirmando que GDS se encuentra instalado y disponible.

Comando:

```
RETURN gds.version() AS versionGDS;
```

Respuesta:

```
"2026.10.1"
```

Una vez listo esto, se verificó la carga de los datos, obteniendo los siguientes resultados:

```
MATCH (e:Estudiante)
RETURN count(e) AS estudiantes;
```

```
15
```

```
MATCH (c:Casa)
RETURN count(c) AS casas;
```

```
10
```

```
MATCH (:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
RETURN count(r) AS visitas;
```

```
75
```

2. Parte 1: Exploración y consultas Cypher

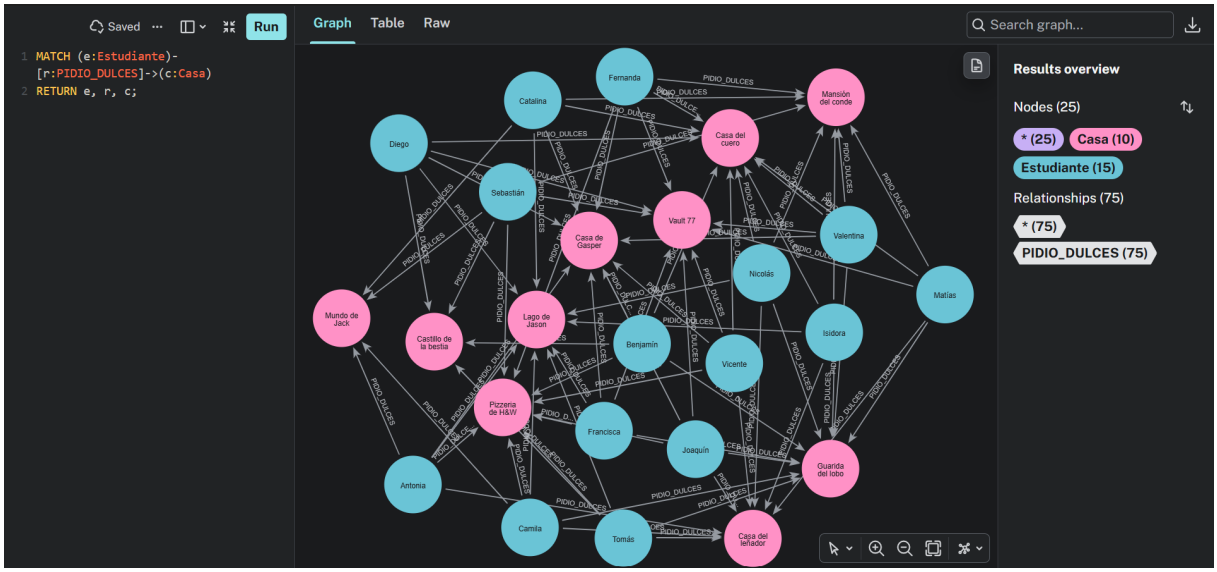
2.1. 1. Visualización e interpretación del grafo

Para la visualización del grafo se utilizó la siguiente consulta:

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
RETURN e, r, c;
```

Explicación de la consulta: Con esta consulta se buscan todas la coincidencias dentro de la base de datos. Con `(estudiante:Estudiante)` se buscan los nodos cuya etiqueta es *Estudiante* y se guardan temporalmente en *e*, con `[relacion:PIDIO_DULCES]->` se buscan las relaciones del tipo *PIDIO_DULCES*, se guardan temporalmente en *r* y con -> se indica que deben ir desde el estudiante hasta la casa, y con `(casa:Casa)` se buscan los nodos con etiqueta *Casa* y se guardan temporalmente en *c*. Finalmente se devuelven los estudiantes, las relaciones y las casas que cumplan este patrón.

Explicación del grafo: El grafo está compuesto por dos tipos de nodos: *Estudiante* y *Casa*. Los estudiantes representan a las personas que participaron en la actividad, mientras que las casas representan los lugares que visitaron. Cada relación *PIDIO_DULCES* es dirigida desde un estudiante hacia una casa y representa una visita realizada. Estas relaciones contienen las propiedades *cantidad*, que indica el número de dulces recibidos, y *tipo*, que identifica el tipo de dulce obtenido. El modelo corresponde a un *grafo bipartito*, ya que sus nodos se dividen en dos conjuntos diferenciados (estudiantes y casas), y las relaciones únicamente conectan elementos de conjuntos distintos, es decir, no existen conexiones entre estudiantes ni entre casas.



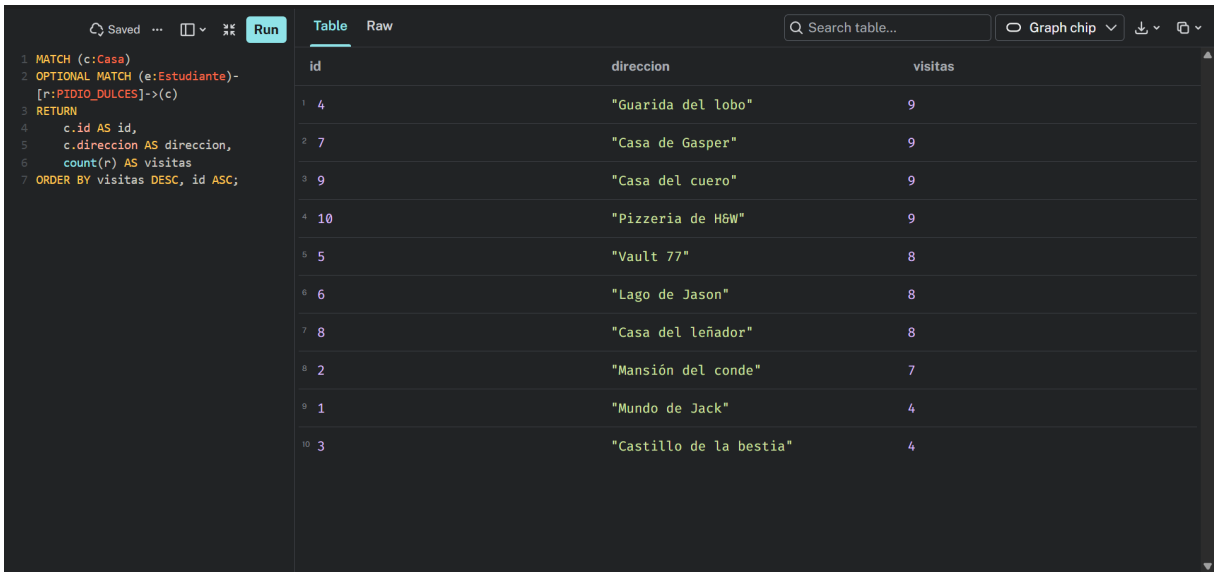
2.2. 2. Casa con más visitas

Para obtener el ranking de las 10 casas se utilizó la siguiente consulta:

```
MATCH (c:Casa)
OPTIONAL MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c)
RETURN
    c.id AS id,
    c.direccion AS direccion,
    count(r) AS visitas
ORDER BY visitas DESC, id ASC;
```

Explicación de la consulta: Primero se obtienen todas las casas, almacenandolas temporalmente en *c*. Luego, con `OPTIONAL MATCH`, se buscan las visitas asociadas, incluso si es que una casa no tuviese visitas. A partir de esto, se cuentan las relaciones encontradas con `count(r)` y se ordena de mayor a menor la cantidad de visitas y para resolver empates, se muestra por id en orden ascendente.

¿Por qué una visita con `cantidad = 0` también se debe contar?: Porque representan situaciones en las que un estudiante visitó una casa, pero no recibió dulces. La cantidad de dulces obtenidos no determina si la visita ocurrió o no, sino que únicamente indica su resultado. Por lo tanto, estas relaciones deben considerarse al calcular el total de visitas, aunque no aporten dulces al total obtenido por el estudiante.



id	direccion	visitas
4	"Guarida del lobo"	9
7	"Casa de Gasper"	9
9	"Casa del cuero"	9
10	"Pizzeria de H&W"	9
5	"Vault 77"	8
6	"Lago de Jason"	8
8	"Casa del leñador"	8
2	"Mansión del conde"	7
1	"Mundo de Jack"	4
3	"Castillo de la bestia"	4

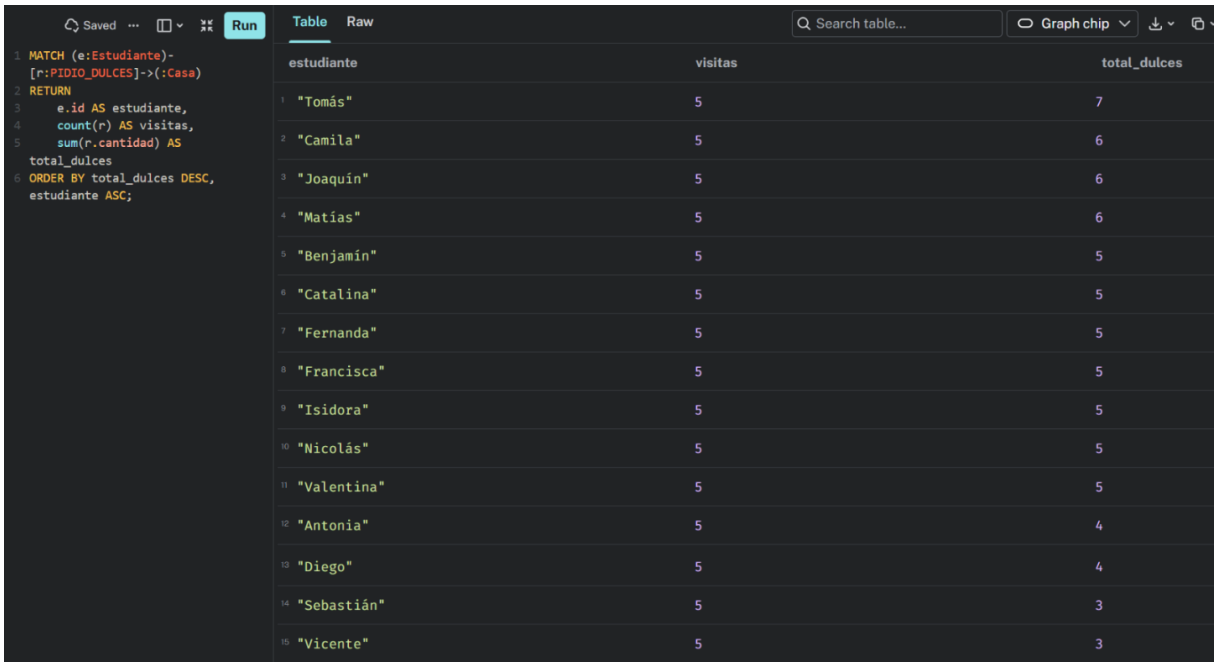
2.3. 3. Estudiante con más dulces

Para obtener el ranking de los 15 estudiantes se utilizó la siguiente consulta:

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
RETURN
    e.id AS estudiante,
    count(r) AS visitas,
    sum(r.cantidad) AS total_dulces
ORDER BY total_dulces DESC, estudiante ASC;
```

Explicación de la consulta: Se buscan los estudiantes y sus relaciones con las casas, y se agrupan las visitas por estudiante. Para contar la cantidad de visitas se utiliza `count(r)` y para sumar los dulces recibidos se utiliza `sum(r.cantidad)`. Finalmente se ordena según el total de dulces de mayor a menor y para empates, de ordena por estudiantes en orden ascendente.

Diferencia entre contar visitas y sumar dulces: Ambas operaciones representan información distinta. Contar visitas permite determinar cuántas casas visitó cada estudiante, considerando todas las relaciones *PIDIO_DULCES*, incluso aquellas cuya cantidad es igual a 0. En cambio, sumar dulces permite calcular la cantidad total de dulces recibidos por cada estudiante, sumando el valor de cantidad en sus visitas. Aunque todos los estudiantes tienen cinco visitas, sus totales de dulces pueden ser diferentes, ya que en cada casa pueden recibir 0, 1 o 2 dulces. Por lo tanto, tener la misma cantidad de visitas no implica haber obtenido la misma cantidad de dulces.



```
MATCH (e:Estudiante)-
[r:PIDIO_DULCES]->(:Casa)
RETURN
    e.id AS estudiante,
    count(r) AS visitas,
    sum(r.cantidad) AS
total_dulces
ORDER BY total_dulces DESC,
estudiante ASC;
```

Table | Raw | Search table... | Graph chip

	estudiante	visitas	total_dulces
1	"Tomás"	5	7
2	"Camila"	5	6
3	"Joaquín"	5	6
4	"Matías"	5	6
5	"Benjamín"	5	5
6	"Catalina"	5	5
7	"Fernanda"	5	5
8	"Francisca"	5	5
9	"Isidora"	5	5
10	"Nicolás"	5	5
11	"Valentina"	5	5
12	"Antonia"	5	4
13	"Diego"	5	4
14	"Sebastián"	5	3
15	"Vicente"	5	3

2.4. 4. Visitas sin dulces

Para obtener las visitas cuya cantidad es 0 se utilizó la siguiente consulta:

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
WHERE r.cantidad = 0
RETURN
    e.id AS estudiante,
    c.direccion AS casa,
    r.tipo AS tipo;
```

Para obtener la cantidad total de visitas sin dulces se utilizó la siguiente consulta:

```
MATCH ()-[r:PIDIO_DULCES]->()
WITH
    count(r) AS total_visitas,
    sum(CASE WHEN r.cantidad = 0 THEN 1 ELSE 0 END)
        AS visitas_sin_dulces
RETURN
    visitas_sin_dulces,
    total_visitas,
    100.0 * visitas_sin_dulces / total_visitas
        AS porcentaje;
```

Explicación de las consultas: Para la primera consulta, se buscan los estudiantes y sus relaciones con las casas para posteriormente mostrar aquellas visitas cuya cantidad es 0. En la segunda consulta, se cuentan todas las visitas realizadas y aparte se suman las visitas cuya cantidad es 0. Finalmente, retora el porcentaje de las visitas sin dulces respecto a las visitas totales.

Comparación de porcentajes: Durante la generación de los datos se utilizó una probabilidad del 20 % para las visitas sin dulces. Sin embargo, el porcentaje observado difiere con dicho valor siendo este un 16 % debido a que los datos se generaron aleatoriamente y la cantidad de observaciones es limitada. Por esta razón, el porcentaje obtenido representa la frecuencia observada en la base de datos y no necesariamente coincide con la probabilidad utilizada durante su generación.



```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
WHERE r.cantidad = 0
RETURN
    e.id AS estudiante,
    c.direccion AS casa,
    r.tipo AS tipo;
```

estudiante	casa	tipo
"Diego"	"Casa del cuero"	"Ninguno"
"Sebastián"	"Pizzeria de H&W"	"Ninguno"
"Antonia"	"Mundo de Jack"	"Ninguno"
"Valentina"	"Mansión del conde"	"Ninguno"
"Sebastián"	"Castillo de la bestia"	"Ninguno"
"Benjamín"	"Castillo de la bestia"	"Ninguno"
"Fernanda"	"Vault 77"	"Ninguno"
"Vicente"	"Vault 77"	"Ninguno"
"Catalina"	"Lago de Jason"	"Ninguno"
"Nicolás"	"Lago de Jason"	"Ninguno"
"Vicente"	"Casa de Gasper"	"Ninguno"
"Francisca"	"Casa de Gasper"	"Ninguno"



```
MATCH ()-[r:PIDIO_DULCES]->()
WITH
    count(r) AS total_visitas,
    sum(CASE WHEN r.cantidad = 0 THEN 1 ELSE 0 END)
        AS visitas_sin_dulces
RETURN
    visitas_sin_dulces,
    total_visitas,
    100.0 * visitas_sin_dulces / total_visitas
        AS porcentaje;
```

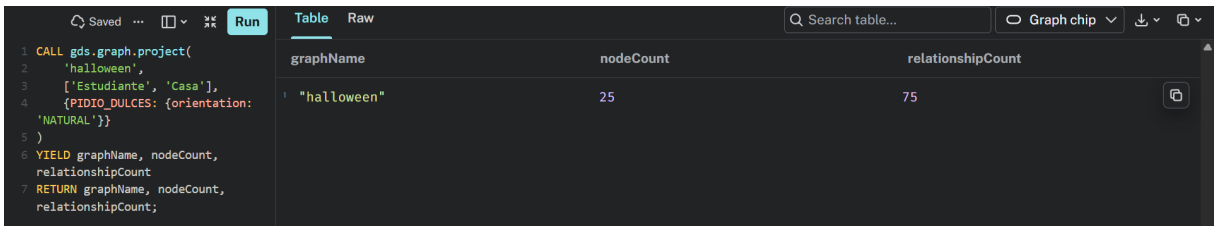
visitas_sin_dulces	total_visitas	porcentaje
12	75	16.0

3. Parte 2: Análisis con GDS

3.1. 1. Proyección del grafo

Proyección llamada 'halloween':

```
CALL gds.graph.project(
    'halloween',
    ['Estudiante', 'Casa'],
    {PIDIO_DULCES: {orientation: 'NATURAL'}}
)
YIELD graphName, nodeCount, relationshipCount
RETURN graphName, nodeCount, relationshipCount;
```



```
CALL gds.graph.project(
    'halloween',
    ['Estudiante', 'Casa'],
    {PIDIO_DULCES: {orientation: 'NATURAL'}}
)
YIELD graphName, nodeCount, relationshipCount
RETURN graphName, nodeCount, relationshipCount;
```

graphName	nodeCount	relationshipCount
"halloween"	25	75

¿Para qué sirve proyectar el grafo y qué significa NATURAL?: La proyección permite crear una representación del grafo en memoria sobre la cual se pueden ejecutar los algoritmos de GDS sin modificar los nodos ni las relaciones originales de la base de datos. En este caso, la proyección halloween incluye los nodos con etiquetas Estudiante y Casa, junto con las relaciones PIDIO_DULCES. El resultado corresponde a 25 nodos y 75 relaciones, que representan los

15 estudiantes, las 10 casas y las visitas registradas, respectivamente. La orientación NATURAL indica que se conserva la dirección original de las relaciones. En este caso, las conexiones van desde los estudiantes hacia las casas, ya que representan las visitas que cada estudiante realizó.

¿Por qué no se necesita proyectar cantidad para los análisis sin pesos?: No es necesario proyectar la propiedad cantidad porque se considera cada visita como una conexión independientemente de la cantidad de dulces recibidos, es decir, todas las relaciones tienen la misma importancia, incluso si su propiedad cantidad es igual a 0. Por lo tanto, basta con proyectar los nodos y las relaciones, sin incorporar propiedades que asignen pesos a las conexiones.

3.2. 2. Degree Centrality: centralidad de las casas

La consulta utilizada para calcular cuántas visitas recibe cada casa fue la siguiente:

```
CALL gds.degree.stream('halloween', {
    orientation: 'REVERSE'
})
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS nodo, score
WHERE 'Casa' IN labels(nodo)
RETURN
    nodo.id AS id,
    nodo.direccion AS direccion,
    score AS visitas
ORDER BY visitas DESC, id ASC;
```

Explicación de la consulta: con `gds.degree.stream` se ejecuta el algoritmo *Degree Centrality*, luego con la orientación *REVERSE* se invierte la orientación de las relaciones, con `YIELD nodeId, score` se obtiene el identificador del nodo junto con su puntaje de centralidad, con *gds.util.asNode(nodeId) AS nodo, score* se recupera el nodo original, *WHERE 'Casa' IN labels(nodo)* filtra para solo mostrar casas y finalmente se ordenan las visitas de mayor a menor y el id de menor a mayor para los casos en que se produzca empate.

Comparación de puntaje, explicación de mayor centralidad y uso de REVERSE: Debido a que las relaciones están dirigidas desde los estudiantes hacia las casas, se utiliza la orientación REVERSE para invertir el sentido de recorrido y calcular cuántas relaciones recibe cada casa. Esto es necesario porque al utilizar la orientación NATURAL, las casas tendrían un grado de salida igual a 0, mientras que los estudiantes tendrían un grado de salida igual a 5 ya que cada uno visita exactamente cinco casas. De esta manera, al utilizar REVERSE, el puntaje de centralidad obtenido para cada casa corresponde a su cantidad de visitas recibidas.

Al comparar estos resultados con el ranking obtenido en la Parte 1, se observa que ambos presentan los mismos valores y el mismo orden de casas, ya que los dos procedimientos contabilizan las relaciones asociadas a cada casa. Esto permite comprobar que el análisis mediante Degree Centrality entrega los mismos resultados que el conteo directo de visitas realizado mediante Cypher.

Por otra parte, una mayor centralidad significa que una casa fue visitada por una mayor cantidad de estudiantes. Sin embargo, esto no garantiza que haya entregado más dulces, ya que el algoritmo considera únicamente la cantidad de conexiones y no los valores de la propiedad cantidad de cada relación. Por lo tanto, una casa puede recibir muchas visitas y aun así entregar menos dulces que otra con menos visitas.



Según los resultados obtenidos, las casas *Guarida del lobo*, *Casa de Gasper*, *Casa del cuero* y *Pizzeria de H&W* comparten el mayor puntaje ya que presentan una misma centralidad de 9 visitas; luego *Vault 77*, *Lago de Jason* y *Casa del leñador* tienen una centralidad de 8; *Mansión del conde* una centralidad de 7; y *Mundo de Jack* y *Castillo de la bestia* una centralidad de 4, siendo estos los de menor cantidad de visitas.

3.3. 3. Node Similarity: estudiantes con visitas parecidas

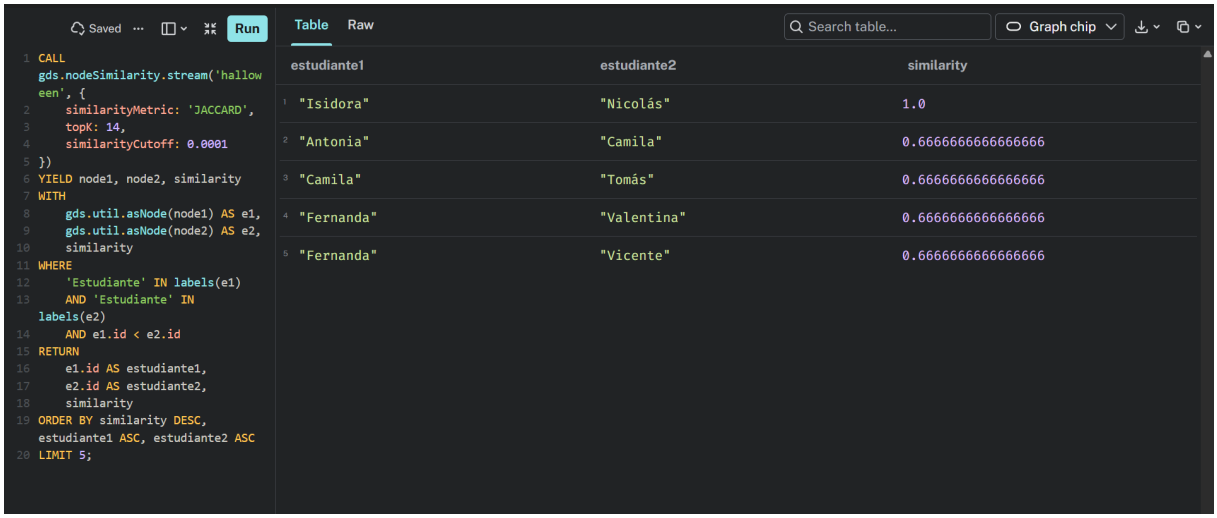
La consulta utilizada para mostrar las 5 parejas distintas con mayor similitud es la siguiente:

```
CALL gds.nodeSimilarity.stream('halloween', {
    similarityMetric: 'JACCARD',
    topK: 14,
    similarityCutoff: 0.0001
})
YIELD node1, node2, similarity
WITH
    gds.util.asNode(node1) AS e1,
    gds.util.asNode(node2) AS e2,
    similarity
WHERE
    'Estudiante' IN labels(e1)
    AND 'Estudiante' IN labels(e2)
    AND e1.id < e2.id
RETURN
    e1.id AS estudiante1,
    e2.id AS estudiante2,
    similarity
ORDER BY similarity DESC, estudiante1 ASC, estudiante2 ASC
LIMIT 5;
```

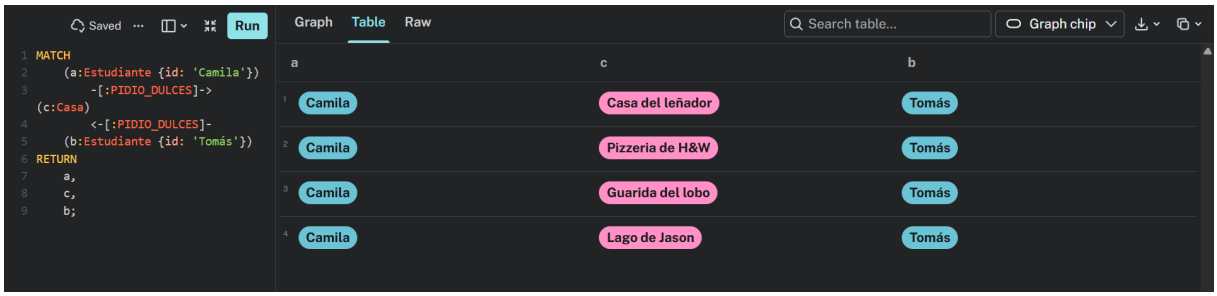
La consulta utilizada para mostrar las casas que ambos estudiantes visitaron es la siguiente:

```
MATCH
    (a:Estudiante {id: 'Camila'})
        -[:PIDIO_DULCES]->(c:Casa)
        <-[:PIDIO_DULCES]-
    (b:Estudiante {id: 'Tomas'})
RETURN
    a,
    c,
    b;
```

Explicación de las consultas: Para la primera consulta, con `gds.util.asNode` se recuperan los nodos correspondientes y se asignan a las variables `e1` y `e2`. Con `WHERE` se conservan únicamente las parejas formadas por estudiantes, descartando otros tipos de nodos. Con la condición `e1.id <e2.id` se evita que una misma pareja aparezca dos veces en distinto orden y finalmente, se muestran los nombres de ambos estudiantes junto con su similitud, ordenando los resultados de mayor a menor puntaje y utilizando los nombres como criterio secundario de ordenamiento. Con `LIMIT 5` se obtienen las cinco parejas distintas con mayor similitud. Para la segunda consulta, se busca un patrón que conecte a los estudiantes Camila y Tomás con una misma casa, de esta forma se identifican únicamente las casas que fueron visitadas por los dos estudiantes. Finalmente, se muestran ambos estudiantes y las casas que tienen en común.



Pareja seleccionada y casas visitadas en común: Se seleccionó la pareja formada por Camila y Tomás. A través de la consulta Cypher se identificaron cuatro casas que ambos estudiantes visitaron: Casa del leñador, Pizzeria de H&W, Guarida del lobo y Lago de Jason.



Para el cálculo de la similitud de Jaccard se sabe que Camila y tomás visitaron 5 casas y que ambos visitaron 4 en común, por lo tanto, la fórmula queda como:

$$J(A,B) = \frac{|V(A) \cap V(B)|}{|V(A) \cup V(B)|}$$

$$J(\text{Camila}, \text{Tomás}) = \frac{4}{5+5-4} = \frac{4}{6}$$

$$J(\text{Camila}, \text{Tomás}) = 0{,}6667$$

El valor obtenido manualmente coincide con el puntaje similarity que entregó GDS para Camila y Tomás, considerando diferencias de redondeo. Esto se debe a que ambos métodos utilizan la misma fórmula, con la diferencia de que GDS la calcula a partir del grafo mientras que manualmente se calcula a través de la identificación de las casas en común.

¿Qué representa un resultado cercano a 1 y por qué visitar casas similares no implica haber obtenido la misma cantidad o tipo de dulces?: Un resultado cercano a 1 indica que los estudiantes visitaron conjuntos de casas muy similares. Esto no significa que hayan recibido la misma cantidad o tipo de dulces, ya que Node Similarity compara únicamente las casas visitadas y no considera las propiedades cantidad ni tipo de las relaciones.

4. Parte 3: Búsqueda de caminos

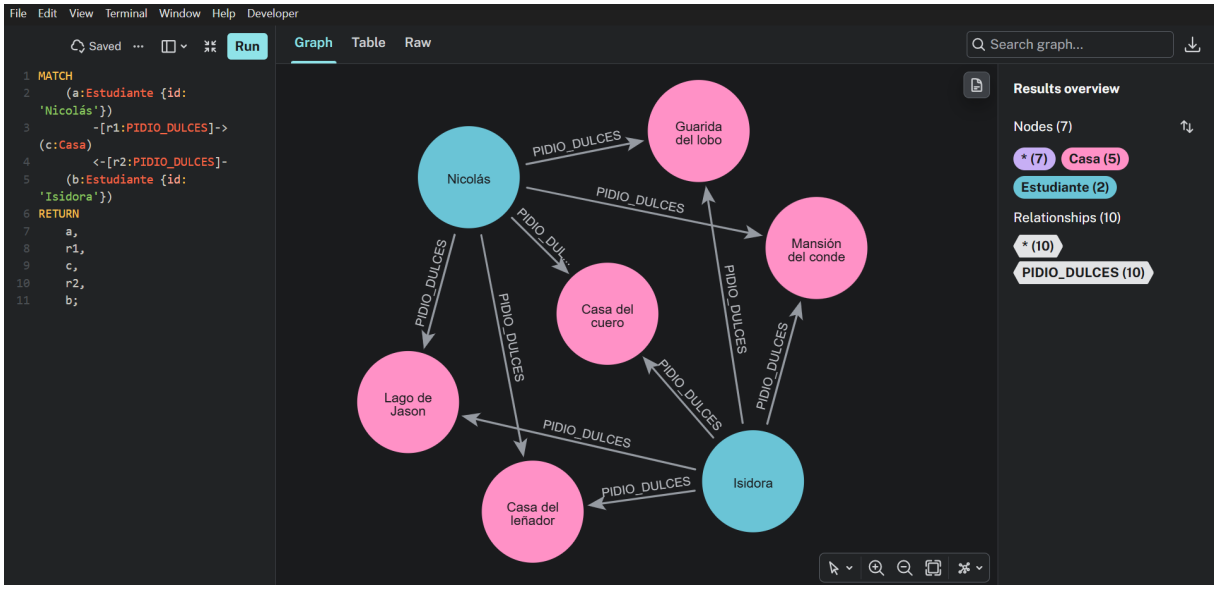
4.1. 1. Casas compartidas

```
MATCH
    (a:Estudiante {id: 'Nicolas'})
        -[r1:PIDIO_DULCES]->(c:Casa)
        <-[r2:PIDIO_DULCES]-
    (b:Estudiante {id: 'Isidora'})
RETURN
    a,
    r1,
    c,
    r2,
    b;
```

¿Por qué cada casa compartida produce un camino de longitud 2 entre ellos?: Porque para conectar a los dos estudiantes necesitamos recorrer dos relaciones: 1. Desde el primer estudiante hacia la casa, y 2. Desde la casa hacia el segundo estudiante. Por lo tanto,

$$A \to C \to B$$

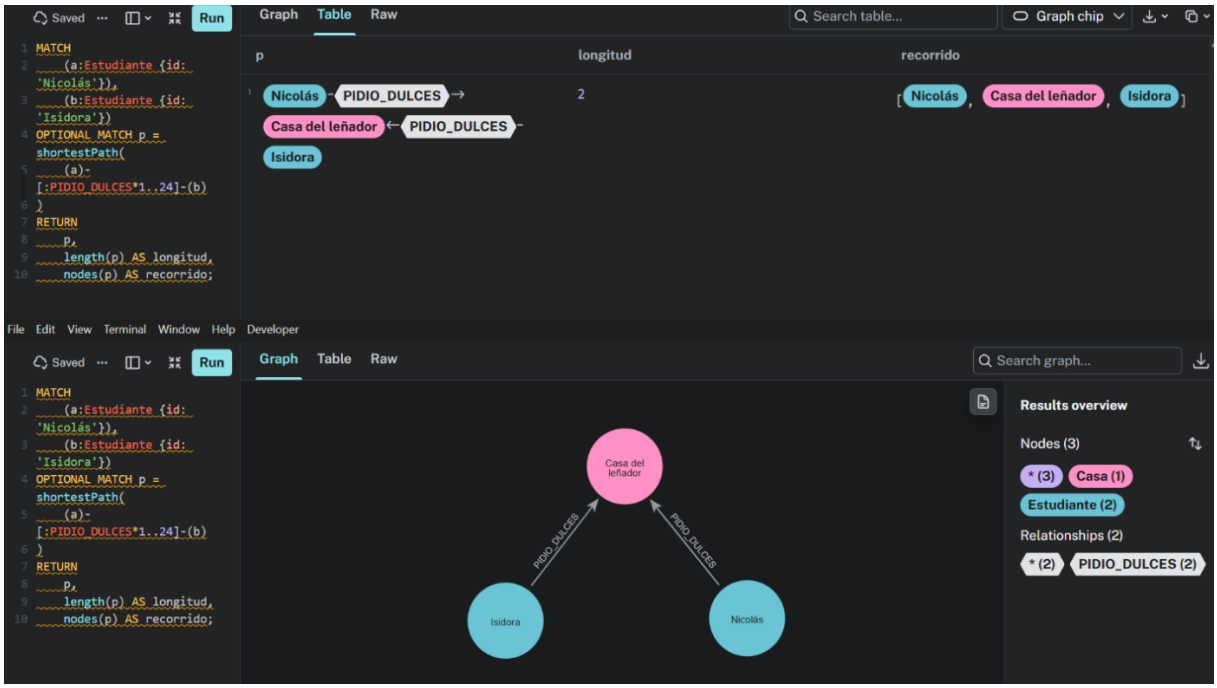
tiene dos relaciones, es decir, longitud 2.



4.2. 2. Camino más corto

```
MATCH
    (a:Estudiante {id: 'Nicolas'}),
    (b:Estudiante {id: 'Isidora'})
OPTIONAL MATCH p = shortestPath(
    (a)-[:PIDIO_DULCES*1..24]-(b)
)
RETURN
    p,
    length(p) AS longitud,
    nodes(p) AS recorrido;
```

El camino más corto obtenido entre los estudiantes Nicolás e Isidora tiene una longitud de 2. Esto se debe a que el grafo conecta a los estudiantes mediante las casas que visitaron, sin existir relaciones directas entre estudiantes. Además, se verificó que los estudiantes comparten al menos una casa entre sí, por lo que siempre existe un camino compuesto por dos relaciones: desde el primer estudiante hacia una casa en común y desde esta hacia el segundo estudiante.



4.3. 3. Interpretación

El recorrido alterna entre estudiantes y casas debido a que el grafo es bipartito, es decir, está compuesto por dos conjuntos de nodos distintos (estudiantes y casas), donde las relaciones `PIDIO_DULCES` conectan únicamente nodos de conjuntos diferentes. Por esta razón, para conectar a dos estudiantes, el camino debe pasar necesariamente por una o más casas.

Al buscar el camino más corto se ignora la dirección de las relaciones ya que se utiliza una relación sin orientación en la consulta (`-[:PIDIO_DULCES]-`). Esto permite recorrer las conexiones en ambos sentidos, aunque originalmente estén dirigidas desde los estudiantes hacia las casas. Por otra parte, la longitud del camino corresponde al número de relaciones que se recorren entre los nodos, y no a la cantidad de dulces obtenidos, pues, cada relación representa una visita de un estudiante a una casa, independientemente de la cantidad de dulces recibidos.

Finalmente, este recorrido representa las conexiones entre estudiantes a través de las casas que tienen en común, por lo que no corresponde a una ruta geográfica ni permite determinar el orden cronológico en que se realizaron las visitas.

5. Cierre

5.1. 1. Reflexión

Los resultados obtenidos mediante Cypher, GDS y el cálculo de caminos permiten analizar las conexiones entre estudiantes y casas desde distintas perspectivas. Por una parte, mediante Cypher se identificó que Camila y Tomás visitaron cuatro casas en común y se identificó cuales fueron, lo que permite conocer directamente las conexiones compartidas entre ambos estudiantes. Por otra parte, el algoritmo Node Similarity de GDS permitió comparar los estudiantes según las casas que visitaron, identificando parejas con conjuntos de visitas similares mediante la similitud de Jaccard, como por ejemplo, Isidora y Nicolás que cuentan con una similitud de 1.0. Finalmente, el cálculo del camino más corto mostró que la conexión entre dos estudiantes puede realizarse mediante una casa compartida, obteniendo una longitud de 2 relaciones. En conjunto, estos resultados permiten comprender cómo se relacionan los estudiantes a través de sus visitas, sin embargo, una limitación de este modelo es que representa las visitas mediante conexiones entre estudiantes y casas, pero no considera información geográfica ni el orden cronológico de estas visitas. Además, aunque registra la cantidad y el tipo de dulces recibidos, estos atributos no son considerados por el algoritmo Node Similarity o por el algoritmo Degree Centrality.

5.2. 2. Liberar la proyección

Evidencia de la liberación de la proyección:

